No of Pages :5 **Course Code : 23MX23**

Roll No:
(To be filled in by the candidate)

PSG COLLEGE OF TECHNOLOGY, COIMBATORE - 641 004
SEMESTER EXAMINATIONS, JUNE 2025
MCA Semester :2
23MX23 OBJECT ORIENTED PROGRAMMING USING JAVA

Time : 3 Hours **Maximum Marks :100**

INSTRUCTIONS:

1. Answer **ALL** questions. Each question carries 25 Marks.
2. Course Outcome : Table

Qn.1	CO1	Qn.2	CO2	Qn.3	CO3	Qn.4	CO4

COs	**At the end of learning this course, students will be able to**
CO1	Comprehend object oriented concepts and fundamentals of Java for problem solving.
CO2	Write applications using Object Oriented features with proper exception handling mechanisms.
CO3	Write programs using streams and multithreading.
CO4	Design and develop applications with user interfacing, Collections framework and database connectivity.

Question **(BTL)**

1. a) Consider the following Java code: (3) (L5)

```
public class myArray{
 public static void main(String[] args) {
  int[] array1 = {10, 20, 30};
  int[] array2 = array1;
  array2[0] = 99;
System.out.println("array1[0]: " + array1[0]);
  array1 = new int[] {5, 6, 7};
  array2[0] = 999;
System.out.println("array1[0]: " + array1[0]);
  }
}
```

i) What will be the output of the above program?

ii) Explain why this output occurs, focusing on the concept of references in Java arrays.

b) i) Explain key difference between String and StringBuilder in Java in terms of memory and performance. What will be the output of the following code? Justify your answer. (3) (L3)

```
public class StringTest {
   public static void main(String[] args) {
      String str = "Hello";
StringBuildersb = new StringBuilder("Hello");
str.concat(" World");
sb.append(" World");
System.out.println("String: " + str);
System.out.println("StringBuilder: " + sb);
   }
}
```

ii) Can a static method directly access instance variables or call non-static methods? Define and clearly differentiate variable and methods declared as static and which is not static. State the key characteristics and purpose of static methods by giving the reasoning for when and why are they used. Justify your answer with a real-world example where using a static method is preferable than an instance method. (7) (L3)

c) Exemplify the abstraction, encapsulation and inheritance in object oriented programming through the use of access specifiers. (4x3=12) (L5)

Create a class person with the following attributes.

name (public)

age (protected)

socialSecurityNumber (private)

address(default)

static count

i) Create a subclass Employee that extends Person.

ii) Which attributes are directly accessible within the class, subclass, and from outside the package?

iii) How does this example demonstrate abstraction, encapsulation, andinheritance?

iv) How does the static variable behave across multiple objects?

2. a) i) In Java, if base class and its sub class is having same data member by name, how will you bring the scope of the base class data member in sub class? Provide a code snippet. (L3)

ii) Consider the following code:

```
public final class Test {
    final int x=10;
    ... // code continues from here
```

In this context, what does the word final signify about the class Test?

iii) What is the significant use of abstract class? (3)

b) i) Define checked and unchecked exceptions in Java. Name few exception under each category. (3) L2

ii) Consider the following code and analyze its output. Answer the following questions. (7) L4

```
public class ExceptionDemo {
  public static int test(int a, int b) {
    if (a < b) {
      return 0;
    } else {
      return 1 + test(a - b, b);
    }
  }
public static void main(String[] args) {
try {
int result = test(15, 4);  // Line A
```

```
System.out.println("Result: " + result);
//assume your project folder has only  java files.
FileReader file = new FileReader("data.txt"); // Line B
} catch (FileNotFoundException e) {
System.out.println("FileNotFoundException caught");
 }
catch (ArithmeticException e) {
System.out.println("ArithmeticException caught");}
catch (Exception e) {
System.out.println("General Exception caught");}
System.out.println("Program ends"); }
}
```

w) What output will this program produce?

x) What type of exception is thrown at runtime, and why?

y) Does the program ever reach Line B? Explain your reasoning.

z) Rewrite the test method so that it throws an IllegalArgumentException when b == 0.

c) i) Explain why Java does not support multiple inheritance with classes.What alternative mechanism does Java provide to achieve multiple inheritance behaviour?How does Java resolve ambiguity when a class implements multiple interfaces with the same default method? (12) L5

Illustrate with the following scenario:

- An interface Writer with a method write().
- An interface Speaker with a method speak().
- A class Person that implements both Writer and Speaker and provides concrete implementations of both methods.
- Modify Writer and Speaker to include a default method communicate() with different messages.In the Person class, resolve the ambiguity by overriding the communicate() method and calling both interface versions inside it.

(OR)

ii) Describe the significance of overloading and overriding with real time examples. Discuss how **overloading and overriding** differ in terms of: Binding time (compile-time vs runtime), Inheritance requirement and Parameter list. Design a class hierarchy that uses **runtime polymorphism** using **method overriding**. Define a **base class** called Payment with a method void processPayment(double amount) that print ”generic payment of $<amount>". Create two subclasses:

CreditCardPayment that **overrides** processPayment() and prints: “credit card payment of $<amount>"

UPIPayment that **overrides**processPayment() and prints:”UPI payment of $<amount>"

3. a) Consider the following Java program: (3) (L3)

```
class MyThread extends Thread {
   public void run() {
for (inti = 0; i< 3; i++) {
```

```
System.out.println(Thread.currentThread().getName()
+ " - " + i);  }    }
}
public class ThreadTest {
   public static void main(String[] args) {
MyThread t1 = new MyThread();
MyThread t2 = new MyThread();
      t1.start();
      t2.start();
System.out.println("Main thread ends"); }}
```

Now, modify the main method to include t1.join(); and t2.join(); as shown:

```
 t1.start();
      t1.join();
      t2.start();
      t2.join();
```

i) What is the difference in output between the original and modified versions of the program?

ii) How join() changes the behaviour of the threads?

iii) Why might you use join() in real-world multithreaded applications?

b) i) State the importance of object serialization. Model a currency converter with member functions to convert rupees to dollars and dollars to rupees. Create an object for converter and write it to a file “data.txt”. Also write a program that reads an object from the file. (3) L3

ii) Compare the reader/writer class hierarchy and the inputStream/ outputStream class hierarchy. Can we say the file streams in java works on the principles of object serialization? Justify your answer. (7) L4

c) Draw and explain the life cycle of a thread in Java. List the key methods associated with each stage of the thread life cycle. Write a Java program that Accepts 10 numbers as input from the user. Use multithreading to compute and display sum of the even and odd numbers separately using the sleep()and join()methods in your code snippet. (12) L4

4. a) What is the type of copy performed in this code (shallow copy or deep copy)? State the reasons why changing emp2's values does not affect emp1. (3) (L3)

```
public class TestClone {
   public static void main(String[] args) throws
CloneNotSupportedException {
      Employee emp1 = new Employee("John", 101);
      Employee emp2 = (Employee) emp1.clone();
      emp2.name = "Alice";
      emp2.id = 102;
System.out.println("Employee 1: " + emp1.name + ", ID: " +
emp1.id);
System.out.println("Employee 2: " + emp2.name + ", ID: " +
emp2.id);}
}
```

b) i) State the purpose of using an ArrayList in Java. Explain how using **generics** with ArrayList improves type safety and code reusability. Define a generic class DataStore<T> that uses an ArrayList to store elements of any type.Includes methods to add an element, display all elements, and search for a specific element.Design a proper driver class to demonstrate the class by creating a DataStore<String> and a DataStore<Integer> and performing operations on both. (5) (L5)

ii) Exemplify the collection framework with significant the classes and interfaces available. Design an online dictionary application, which allows the user to search online dictionary and get the meaning of the words. Use collections for organizing words and meanings. (5) (L5)

c) i) Explain the Event Delegation Model used in Java for event handling. How does it differ from the traditional event-handling model? Implement a simple Java GUI application using AWT or Swing where a button click updates a label with a message “Successful end of Year 1” using the Event Delegation Model in your code. (12) L5

(OR)

ii) Exemplify the role and advantages of PreparedStatement in JDBC. How does it differ from a Statement?

Design a Java program that connects to a MySQL database named studentdb and inserts student records into a table named students. The table has the following structure:

CREATE TABLE students(id INT PRIMARY KEY, name VARCHAR(50), grade FLOAT);

Write a Java program using preparedStatement to:

- Connect to the database.
- Take input for student id, name, and grade.
- Insert the record into the students table.
- Ensure resources are closed properly.

CO : COURSE OUTCOME; BTL : BLOOM’S TAXONOMY LEVEL

/END/

FD/RL